

随着硬件技术的飞速发展,内存数据库正成为当前数据库的主流技术,在高性能事务处理和高性能分析处理领域发挥着重要的作用。从数据库技术的发展路线来看,存储器与处理器技术的升级推动数据库实现了从磁盘数据库到内存数据库(IMDB)的技术升级,而存储设备与处理器的硬件特性也成为新型数据库存储引擎和查询引擎设计的重要因素。

第 14 章 14.3.2 小节已初步介绍了内存数据库的概念,本章将在其基础上简要介绍与内存数据库密切相关的硬件技术及发展趋势、内存数据库主要的实现技术,以及内存数据库的几种实现方案,使读者对新硬件时代数据库技术的发展有初步的了解。

17.1 内存数据库概述

内存数据库是系统将内存作为主存储设备的数据库系统。内存数据库的数据组织、存储访问模型和查询处理模型都是针对内存特性而优化设计的,内存数据被处理器直接访问,磁盘只是作为后备存储设备使用,并不作为系统优化设计的重点。

传统的磁盘数据库(disk resident database, DRDB)使用磁盘作为主要的数据存储设备,使用内存缓冲区作为磁盘数据的高速缓存,以提高查询处理性能。磁盘数据库的数据组织、存储和访问模型及处理模型都是面向磁盘访问特性而设计的,磁盘数据通过缓冲区被 CPU 间接访问,缓冲区的效率是查询处理性能的重要影响因素。

图 17.1 为磁盘数据库与内存数据库的对比^[1]。在磁盘数据库中,数据库的基础数据结构,如表、索引、临时文件等都存储在磁盘中。数据存储结构面向磁盘存储结构而设计,数据库的查询优化技术以提高磁盘数据的 I/O 访问性能为中心。因此,在磁盘数据库中,磁盘 I/O 是最重要的性能瓶颈,缓冲区管理是提高查询处理性能的重要技术。通过缓冲区管理优化技术提高频繁访问数据在缓冲区中的命中率,降低 I/O 延迟。

与磁盘数据库相对,内存数据库的数据文件,如表、索引、临时文件等全部驻留于内存,其数据文件的组织采用面向内存访问特点而优化的数据结构,与磁盘数据库基于 I/O 优化的 page-slot 结构有较大的差异。由于内存数据库默认数据驻留于内存,因此不需要磁盘数据库的

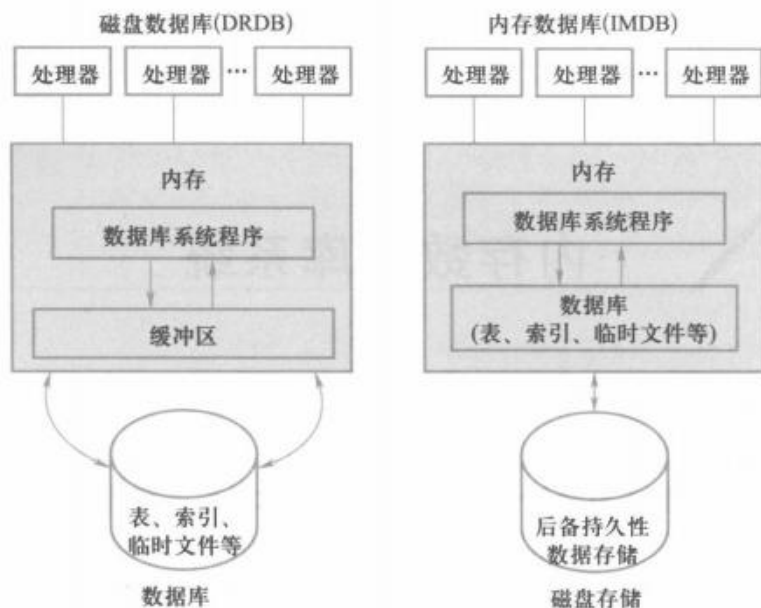


图 17.1 磁盘数据库和内存数据库对比

缓冲区管理机制，数据库系统程序直接访问内存数据结构，能够更加有效地提高数据访问效率。内存数据库的查询优化技术以内存数据访问、Cache 优化、多核并行优化等为核心。

磁盘数据库的缓冲区是磁盘数据在内存的副本，采用与磁盘存储一致的基于 page-slot 的数据结构，当内存足够将全部磁盘数据缓存在缓冲区时，数据访问只是相当于在内存盘(RAM disk)上的访问。内存数据库采用面向内存访问特点的数据组织结构，其数据访问性能仍然优于全部缓存模式下的磁盘数据库。它并不是简单地将磁盘数据全部缓冲到内存，而是面向内存存储访问特点和多核处理器并行计算特点进行全面优化设计的新的数据库系统，需要对传统的数据库理论和实现技术进行全面升级或重新设计。

内存数据库消除了磁盘数据库中巨大的 I/O 代价。同时，数据的存储和访问算法以内存访问特性为基础，实现处理器对数据的直接访问，在算法和代码效率上高于以磁盘 I/O 为基础的磁盘数据库。在内存数据库中，使用针对内存特性进行优化的 B+树索引^[2]、T 树索引^[3]和哈希索引、面向 Cache 优化的查询处理算法^[4]和多种面向连接操作的优化技术^[5]，进一步优化了内存数据库的性能。因此，与数据全部缓存到内存的磁盘数据库相比，内存数据库的性能仍然超出数倍。

由于内存是易失性存储介质，因此内存数据库与磁盘数据库相比，在事务的 ACID 特性上能够满足 ACI 特性，但 D 特性的满足需要借助特殊的硬件设备、系统设计和实现机制。例如：

① 日志。内存数据库需要在事务提交前将日志写到可靠存储设备上，可靠存储设备的访问性能将影响内存数据库的性能。

② 检查点。检查点周期性地将内存数据记录到磁盘上，在发生系统故障并进行恢复时能

够还原某一时刻的数据,需要和日志配合使用来保证数据的一致性。

③ 非易失性存储器(NVM)日志存储。内存数据库使用一些新兴的高性能非易失性存储器作为低延迟日志存储设备,以提高事务处理性能,包括闪存(flash memory)、相变存储器(phase change memory, PCM)^①、非易失性(non-volatile)磁性随机存取存储器(magnetic random access memory, MRAM)、3D XPoint 非易失性内存等。

④ 高可用性(high availability)技术。使用数据库复制技术,采用主-备双机内存数据库服务器,在发生故障时数据库系统能够自动在数据库副本之间进行切换,实现不间断服务,提高数据库可靠性。

内存数据库一般用于对实时响应性要求较高的高端应用领域,如电信、金融等领域的核心事务处理。内存数据库既可以作为独立的高性能数据库来处理核心业务,也可以作为磁盘数据库的高速缓存,加速磁盘数据库中热数据集的处理性能。在后一种应用模式中,需要对数据库的模式进行优化,划分出热数据集和冷数据集,由内存数据库和磁盘数据库来分别处理,在两个数据库之间通过数据迁移技术实现底层数据的融合。

将内存数据库运行在大内存、多级 Cache 和多核硬件环境下,还可以有效解决计算密集型的联机分析处理应用的性能瓶颈。这类分析型内存数据库需要重点解决的问题包括存储模型优化技术、查询处理模型优化技术、轻量压缩技术、Cache 优化技术、多核并行查询处理优化技术、Cache 分区优化技术等。根据分析型数据的特点,分析型内存数据库一般采用列存储技术和轻量数据压缩技术来提高内存存储效率和访问效率,在连接操作中优化内存带宽和 Cache 性能。

随着内存集成度提高、容量增大和成本降低,高性能内存事务处理和分析处理将成为实现实时数据处理的关键技术。近年来的发展趋势是将二者融合在一个统一的内存数据库框架之内^[6,7],为用户提供统一的事务处理与分析处理平台。

17.2 新硬件技术推动内存数据库技术发展

随着新型处理器和存储器技术的发展,计算机的存储层次增多,各存储层次之间在容量、成本、性能等方面差异显著。为充分发挥新硬件的性能优势,内存数据库针对这种结构上的物理差异研究设计了内存数据库的实现和优化技术,从而使算法能够更好地利用存储高层的性能优势和低层的成本优势,提高查询处理性能。

本节概要介绍与内存数据库实现技术密切相关的硬件技术。通过对硬件存储访问及计算特性的分析,理解内存数据库如何通过软硬件一体化设计与优化技术来提高查询处理性能。

17.2.1 多核处理器

多核处理器是指在一块处理器中集成两个以上完整的核心(core),它提供了片内并行处理能

^① 《计算机科学技术名词》第3版中译为 phase change random access memory。

力。多核处理器性能的主要影响因素包括核心处理能力、多级 Cache 结构、内存访问性能等。

现代处理器支持单指令多数据流(SIMD)指令集以向量为单位执行数据处理(如 Intel AVX-512 支持的 SIMD 向量长度为 512 位),使用单条指令对整个向量进行操作。内存数据库广泛利用现代处理器的 SIMD 向量处理技术提高数据处理性能。通过片内集成技术,多核处理器还具有强大的多线程并行处理能力。内存数据库在算法设计中充分考虑现代处理器的指令级并行、SIMD 数据级并行和多核多线程任务级并行处理能力,通过优化设计的并行处理算法提高查询处理性能。

由于处理器和内存性能之间存在一定的差距并呈现逐渐扩大的趋势,处理器通常采用多级 Cache 机制优化数据访问的局部性,提高内存数据访问性能。Cache 容量自上而下增大,访问延迟也依次增大。在内存数据库底层算法的优化设计中,提高数据的 Cache 访问局部性是一项重要的优化技术。

多路处理器通常采用非均匀存储器访问(NUMA)架构,与处理器直接相连的是本地内存,通过处理器间的 QPI 通道可以访问远程内存,每个处理器访问本地内存的延迟低于访问远程内存。在内存数据库的算法设计中,除了需要优化数据在核心内的 Cache 访问局部性之外,还需要优化处理器本地内存访问的局部性,减少跨 NUMA 节点的高延迟内存访问。

多核处理器是内存数据库的基础平台,内存数据库在底层算法设计与优化技术上需要结合多核处理器的硬件特性,通过循环展开、预取、单指令多数据流、并发多线程等技术提高查询时的并行处理能力;同时需要面向多级 Cache 结构,通过优化内存数据访问的数据结构和算法设计提高查询处理时的 Cache 访问局部性,通过高速 Cache 加速内存访问性能,提高内存数据库查询处理的综合性能。

17.2.2 图形处理器

随着图形处理器(GPU)对复杂运算的支持以及可编程性和功能的扩展,GPU 在单精度浮点处理能力和存储器带宽性能上已大大超过 CPU。

CPU 是通用处理器,需要处理复杂的指令和逻辑控制。CPU 主频速度与内存速度之间巨大的访问延迟差异,使其必须依赖大容量缓存来减少内存访问延迟。因此在 CPU 芯片内只集成了较少的算术逻辑运算单元 ALU,而将大量的晶体管用于支持大容量缓存。

GPU 的流多处理器(streaming multiprocessor, SM)内部由大量核心组成。从 GPU 硬件的发展趋势来看,GPU 核心数量增长较快,Cache 容量逐渐增大,核心处理能力不断增强,设备之间的通信能力与性能不断提升,显存容量持续增长。基于 CPU-GPU 的异构计算平台是当前主流的高性能计算平台架构,也是高性能内存数据库的新平台,为内存数据库提供了更加强大的 GPU 加速能力。

近年来,通用图形处理器(general purpose graphic processing unit, GPGPU)以其强大的并行计算性能成为高性能计算的新平台,在 GPGPU 上实现数据库技术^[8,9]也成为高性能数据库的一个研究方向。GPU 数据库通常采用 CPU-GPU 的异构并行结构,即 CPU 负责逻辑性较强的事务处理和流程控制,GPU 负责大量数据的关系操作。数据库中基础的关系操作,如选择、

投影、连接、聚集、排序、索引等操作均已在 GPU 上实现,在 CPU-GPU 混合平台上创建的代价模型辅助数据库在两种平台之间分配不同的关系操作。GPU 数据库将 GPU 作为数据库的加速引擎,为数据库提供一些计算密集型操作的计算加速。随着 GPU 内存容量和内存带宽性能的不断增长,基于 GPU 内存的数据库技术可以提供更高的查询处理性能,在高实时性分析处理领域发挥越来越大的作用。GPU 数据库已成为高性能数据库的代表性技术,尤其在大数据实时分析处理领域得到广泛的关注。

17.2.3 新型非易失性内存

典型的计算机存储层次由内存(以动态随机存取存储器 DRAM 为代表的易失性存储器)和存储器(包括磁盘和固态硬盘)组成,内存支持字节级访问粒度,而存储器基于较大的数据块访问。

基于 3D XPoint 技术的 Intel Optane DC 是一种非易失性存储技术,它支持内存与存储器两种类型,一种是非易失性内存 Optane DC PM,另一种是基于 Optane DC 技术的固态硬盘(SSD)。

Optane DC PM 与 DRAM 相比容量更大,但读/写性能低于 DRAM,在应用中既可以作为大容量内存使用,也可以作为 DRAM 之外独立的非易失性存储设备使用。Optane DC PM 改变了传统存储层次的角色,DRAM 缓存热数据,而 Optane DC PM 作为持久存储的温数据层,提供接近 DRAM 性能的持久性内存存储访问能力,也弥补了内存数据库在 ACID 特性中对持久性 D 支持能力的缺失。

基于 Optane DC 技术的 SSD 与 SSD、磁盘一起构成新的存储层。非易失性内存的大容量和相对于 DRAM 的低成本也进一步降低了内存数据库的存储成本,推动内存数据库成为大数据、低成本、高性能的数据管理和分析处理平台。

新型非易失性内存的出现,改变了传统内存易失性的基本假设,为实现以内存为中心的统一存储结构与存储引擎提供了可能,对内存数据库技术的发展有着重要的推动作用。

随着 Optane DC PM 的推出,数据库厂商已开始研发基于非易失性内存的数据库实现技术。但非易失性内存相对于传统 DRAM 在读/写性能上的差距以及 DRAM-非易失性内存混合存储架构,还需要通过定制化的优化技术才能更好地发挥其性能。另一方面,非易失性内存存在产品化方面存在一定的不确定性,但对内存数据库而言,非易失性内存是提高内存数据库性价比的重要技术支持。

17.3 内存数据库的若干关键实现技术

通用的内存数据库管理系统要为用户提供 SQL 接口,具有内存存储管理、面向内存的查询处理和优化等基本模块,还应提供传统磁盘数据库管理系统的功能,如多用户的并发控制、事务管理和访问控制,能够保证数据库的完整性和安全性,在内存数据库出现故障时能够对系统进行恢复,等等。

内存数据库的性能受硬件特性影响较大,需要研究基于新的硬件体系结构下的实现技术。

下面简要介绍内存数据库中的若干关键技术。

17.3.1 数据存储

内存数据库将全部数据和临时工作数据集驻留在内存，采用面向内存优化访问的存储结构，以优化 CPU 访问和高速缓存(Cache)。

图 17.2 和图 17.3 显示了一个关系表 ORDER 使用磁盘行存储(page-slot)结构和内存列存储结构的示意图。

ORDERKEY ORDERSTATUS TOTALPRICE ORDERDATE

1	O	173665.47	1996/1/2
2	O	46929.18	1996/12/1
3	F	193846.25	1993/10/14
4	O	32151.7	1995/10/11
5	F	144659.2	1994/7/30
6	F	58749.59	1992/2/21
7	O	252004.18	1996/1/10
32	O	208660.75	1995/7/16
33	F	163243.98	1993/10/27
34	O	58949.67	1998/7/21

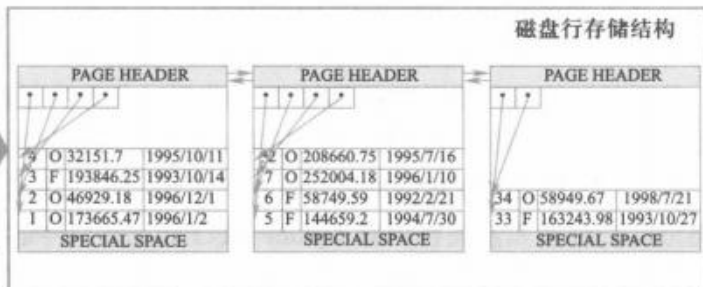


图 17.2 磁盘行存储结构

ORDERKEY ORDERSTATUS TOTALPRICE ORDERDATE

1	O	173665.47	1996/1/2
2	O	46929.18	1996/12/1
3	F	193846.25	1993/10/14
4	O	32151.7	1995/10/11
5	F	144659.2	1994/7/30
6	F	58749.59	1992/2/21
7	O	252004.18	1996/1/10
32	O	208660.75	1995/7/16
33	F	163243.98	1993/10/27
34	O	58949.67	1998/7/21

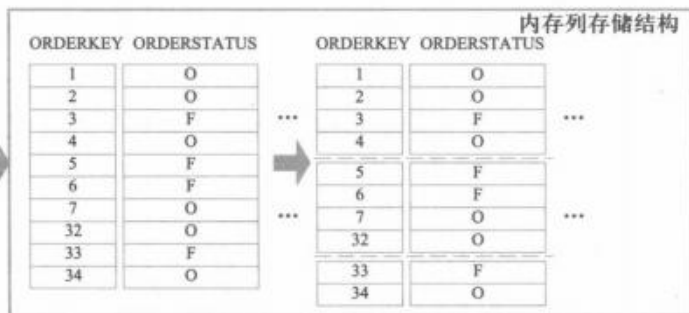


图 17.3 内存列存储结构

在图 17.2 中，磁盘数据库采用适应磁盘 I/O 访问的 page-slot 结构，记录以 slot 结构存储在磁盘页面中，记录通过块号和块内偏移地址进行定位，页面之间创建链表结构。数据库访问时将磁盘页面调入内存缓冲区，记录通过块号和偏移地址来进行查找。

在图 17.3 中，ORDER 表中的数据按列存储，记录可以通过内存指针或列偏移地址直接访问。内存列存储结构适用于数据仓库应用中的分析处理。

进一步地，内存列存储表中的记录可以按一定规则划分为行组，行组内采用列存储，行组内的列可以独立进行压缩。当记录存储为独立的列时，在记录增加时需要重新分配内存空间以容纳新的记录，采用行组结构则可以通过动态增加的行组结构存储新增的记录。

在内存-磁盘(或 SSD)硬件结构中，内存数据库的表、索引等数据结构在内存中采用适合内存访问的结构存储，但还需要存储在磁盘或 SSD 等外部设备上以获得持久数据存储能力，当系统重启时，数据需要从磁盘或 SSD 加载为内存数据结构。

17.3.2 查询处理及优化

内存数据库消除了传统数据库的磁盘 I/O 代价，其代价主要是 CPU 执行代价。CPU 执行代价主要体现在代码执行效率和 Cache 访问效率两个方面。行处理模型、列处理模型、向量化查询处理模型和实时编译查询处理模型是其中比较有代表性的几种查询处理技术，分别从内存访问效率、代码执行效率、Cache 访问效率等方面优化查询处理性能。

传统的查询处理模型是基于行存储结构的迭代处理模型，查询引擎基于语法树迭代解析每条记录以完成查询处理过程。图 17.4 显示了查询 $\text{Price} * \text{Quantity} * (1 - \text{Tax})$ 表达式的不同查询处理模型。

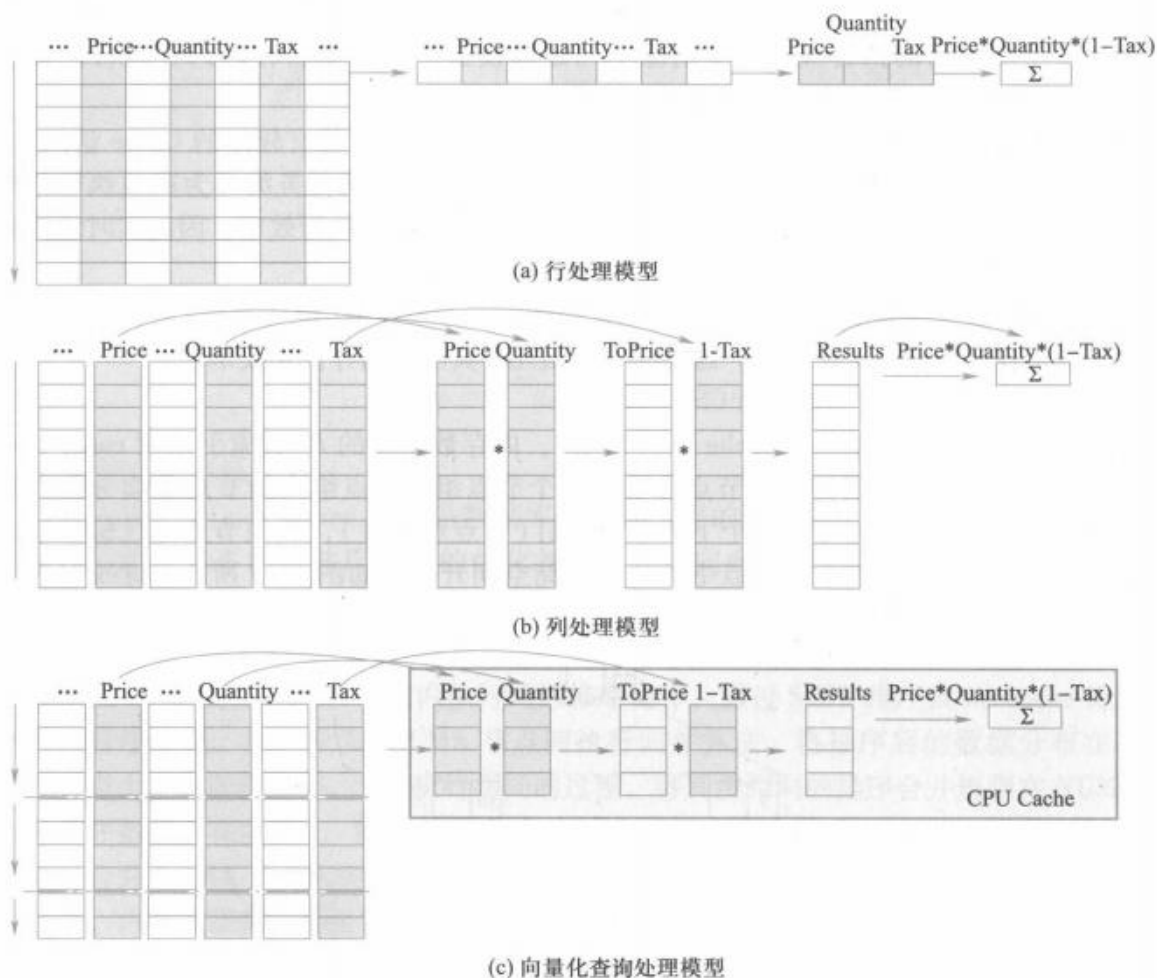


图 17.4 查询处理模型示例

如图 17.4(a) 所示，行处理模型需要查询引擎迭代访问每一条记录，为每一条记录查询关系的元数据，解析查询属性偏移地址并访问查询相关的属性，计算并累加查询表达式。在行处

理模型中,重复执行的记录迭代解析及相应的函数调用执行代价较高。

如图 17.4(b)所示,列处理模型^[10]一次解析并处理整个列,如执行列 Price 和列 Quantity 中所有记录的乘积操作,将中间结果记录在 ToPrice 临时列中。然后再将 ToPrice 列与 Tax 列执行 $ToPrice * (1 - Tax)$ 计算,将列计算结果存储在中间结果列 Results 列中,最后通过对 Results 列中记录的累加计算出查询结果。列处理模型一次解析和函数调用处理全部的列数据,执行代价被大量记录分摊。但一次一列的处理模式需要较大的临时列暂存中间结果数据,产生较大的内存空间消耗和访问代价。

如图 17.4(c)所示,向量化查询处理模型基于 Cache 大小将列数据水平划分为适当大小的向量(如 1 024 个数据作为一个向量),查询引擎以向量为单位执行查询处理任务,记录的解析和函数调用代价被向量中的数据分摊,而且向量化的中间结果列存储于低延迟的 Cache 中,既具有列处理模型较高的执行效率,又消除了列处理模型中间结果暂存的内存代价,实现了 Cache 内的查询处理,提高了查询处理性能。

向量化查询处理模型是当前内存数据库的代表性优化技术,具有较好的 Cache 访问效率。随着编译器技术的发展,现代数据库应用实时编译技术^[11]将查询任务编译为高效执行的机器码或中间级字节码,最小化记录解析与函数调用代价,提高代码执行效率。因此实时编译技术也是现代内存数据库代表性的优化技术。

内存数据库在底层算法实现方面大多采用 hardware-conscious(硬件敏感)的算法设计,基于硬件特性来设计算法与优化技术。这里简单介绍几种典型的内存优化技术。

1. Cache-conscious B+树索引(CSB+树)

内存数据访问的最小单位是 cache line(64 B),内存数据库的 B+树索引采用 cache line 大小的节点存储键值,同一层的索引节点设置为一个节点组,节点组内的节点连续分配内存地址,从而使父节点可以只保存第一个下级节点指针(节点组指针),其他节点通过偏移地址计算获得其内存地址,以节省索引节点中的地址存储空间开销。如图 17.5 所示,虚线框内为节点组,箭头为指向第一个节点的地址指针。

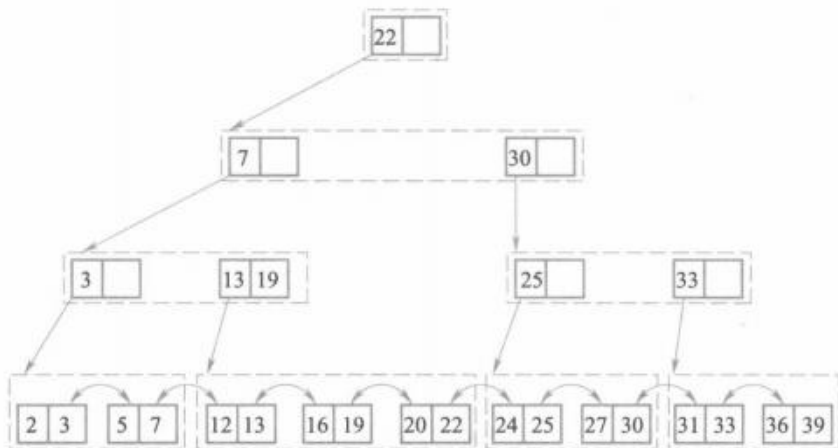


图 17.5 CSB+树示例

2. Radix 分区哈希连接

这种方法通过 Radix 分区技术减少 R 表和 S 表在分区过程中所产生的 Cache 失效和 TLB 失效。如图 17.6 所示, 在分区阶段, 采用 TLB 缓存优化的多趟分区方法将 R 表和 S 表按相同的 Radix 分区算法划分为较小的分区; 在构建哈希表阶段, 各分区上独立创建适合 Cache 大小的哈希表; 在哈希探测阶段, S 表分区探测 R 表分区哈希表, 完成连接操作。基于分区的哈希连接操作在分区阶段需要较大的空间和时间代价, 但分区后的哈希连接能够满足哈希表小于 Cache 容量的要求, 使得在哈希探测阶段获得较好的性能。

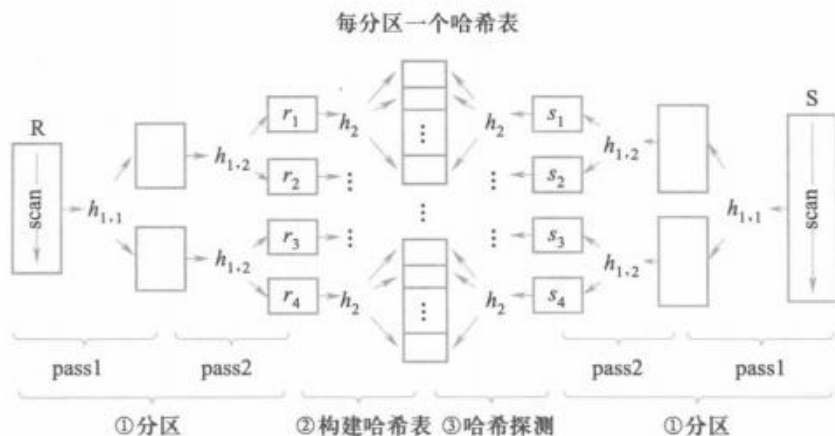


图 17.6 Radix 分区哈希连接

3. 排序合并连接

排序合并连接(sort-merge join)是数据库的一种主要的连接算法。内存数据库的排序合并算法设计需要考虑不同硬件层面上的优化技术, 如面向向量寄存器的 SIMD 排序算法, in-cache 排序算法等。图 17.7 所示的 NUMA-aware 排序合并连接算法^[12]过程优化了基于不同 NUMA 节点内存之间的排序合并连接算法。

在阶段 1, R 表在 NUMA 节点内执行局部排序操作, 通过 SIMD 排序及 in-cache 排序优化本地节点排序性能。在阶段 2, NUMA 节点间执行归并算法, 将排序后的数据分布在不同的 NUMA 节点上。在阶段 3~4, S 表执行相同的过程, 将其按相同的排序合并规则在 NUMA 节点中划分排序数据。在阶段 5, 各 NUMA 节点执行本地排序后 R 表和 S 表分区上的归并连接操作, 最后合并 NUMA 节点间的连接结果。

总之, 内存数据库在算法实现和优化技术上面向现代多核处理器的硬件特性做了深入的优化工作, 如面向 SIMD 的算法实现、面向多线程的并行优化技术、面向多级 Cache 的优化技术等, 以最大化发挥硬件效率, 提高查询处理性能。

随着 GPU、FPGA、Phi 等硬件加速器技术的成熟与应用的普及, 面向 GPU^[13]、FPGA^[14]、Phi^[15] 的内存数据库硬件加速技术成为新的发展趋势。硬件加速器更为强大的并行计算能力, 为内存数据库提供了更高性能的查询加速引擎。

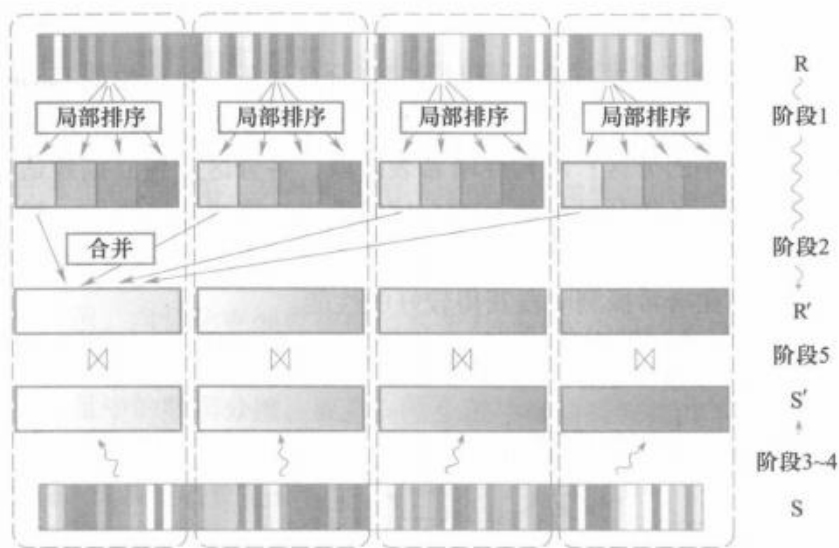


图 17.7 NUMA-aware 排序合并连接算法过程

17.3.3 并发与恢复

1. 并发控制

内存数据库与磁盘数据库的并发控制机制类似，但在细节上存在一定差异。由于数据存储在内存中，内存数据库中的事务执行时间一般较短，因此持锁时间也较短，系统中冲突较少，所以可以采用以下方法减少锁的开销：采用较大的封锁粒度（如表级锁）、采用乐观加锁方式、减少锁的类型、将锁信息存储在数据本身。

对于内存数据来说，封锁产生的 CPU 代价会对性能产生严重的影响，特别是对于工作负载主要由短小事务构成的联机事务处理应用场合，每个事务要求极短的响应时间，在几十毫秒甚至微秒之内完成。针对此问题，S. Blott 等提出了接近串行的并发控制协议。该协议的特点是：写事务在整个数据库上施加互斥锁（mutex），通过时间戳和互斥锁在事务的提交记录没有到达磁盘之前允许新事务开始，并且保证任何提交的读事务不会读到未提交的数据。无锁数据结构（lock-free data structure）也是内存数据库的一项优化技术，通过原子级操作实现无锁并发控制，提高内存数据库的并发查询处理性能。

并发控制会带来一些系统代价，如 CPU 代价、存储代价等，影响系统性能。而内存数据库对性能要求非常高，所以利用内存优势并结合应用需求，在保证事务 ACID 特性的同时尽量减少并发控制对性能的影响，是内存数据库需要进一步研究的问题。

并发查询处理需要解决的问题还包括不同查询任务并发执行时如何高效地解决访问冲突，如联机事务处理与联机分析处理负载混合执行时的并发访问控制问题。Hyper 提出了基于操作系统 snapshot 机制的联机事务处理和联机分析处理混合负载并发执行技术，通过操作系统级的 snapshot 隔离联机事务处理更新数据和联机分析处理只读查询数据，支持不同类型查询任务的

并发执行。

2. 恢复机制

由于 DRAM 内存的脆弱性和易失性, 内存数据库中的数据容易被破坏和丢失, 所以内存数据库数据需要在磁盘等非易失性存储介质中进行备份, 并且在数据更新时将日志写到非易失性存储介质中。

在内存数据库中, 如果在事务提交时将日志写入磁盘, 则由于写日志所产生的磁盘 I/O 会延长事务的处理时间, 降低内存数据库的性能。所以, 将日志写在何处以及何时将日志写入磁盘, 在内存数据库中是一个非常重要的问题。一些研究者提出了预提交、组提交等方法来降低日志 I/O 的代价, 并提出使用 PCM、flash memory 等非易失性内存存储日志的方法。首先将日志存储在非易失性内存中, 然后提交事务, 再异步地把日志写入磁盘。内存数据库的索引构建非常快, 一些内存数据库并不记录索引更新的 redo 日志, 而将 undo 日志保存在内存中, 系统恢复时直接重建索引结构。一些内存数据库通过只记 redo 日志的方式降低日志代价, 通过周期性的检查点技术保证未提交事务数据不写入磁盘, 在系统恢复时加载检查点并重做 redo 日志。也可以通过记录的多版本机制消除记录的原地更新(in-place update), 未提交事务的记录版本最终被垃圾清理进程清除。

在发生系统崩溃时, 如何从备份和日志中恢复数据也是内存数据库重要的性能指标。为了能够尽快地恢复系统的使用, 一般可通过两步来恢复数据: 首先恢复热点数据, 即执行事务所必需的数据, 然后在后台恢复其他非热点数据。另外, 也可根据数据在磁盘上的存储顺序、优先级(是否为热点数据)以及访问频率等参数来确定数据的装载顺序。为了提高内存数据库的恢复性能, 一些内存数据库采用了多核并行恢复技术, 该技术需要将数据和日志进行分区, 以保证分区的日志仅与分区的数据相关, 每个核心可以并行地执行相应数据与日志上的恢复任务。

17.4 内存数据库的几种实现方案

针对不同的应用目标, 内存数据库有不同的实现方案。本节介绍三种主要的方案: 混合的内存加速引擎、独立的内存数据库系统和 GPU 数据库。

17.4.1 混合的内存加速引擎

这种方案是在传统数据库的磁盘处理引擎基础上, 通过集成内存数据处理引擎技术来提升数据库的实时处理能力。当前的发展趋势是混合双/多引擎结构数据库, 图 17.8 为几种产品示例。

图 17.8(a)为 Oracle 公司推出的支持两种存储格式的内存数据库产品 Oracle Database In-Memory, 其行存储结构用于加速内存联机事务处理负载、列存储结构用于加速内存联机分析处理负载。列存储引擎是完全内存列存储结构, 应用 SIMD、向量化处理、数据压缩、存储索

引等内存优化技术,并可以扩展到 RAC 集群提供 Scale-Out 能力和高可用性。

图 17.8(b)为 IBM 公司推出的面向商业智能查询负载的加速引擎 IBM BLU Acceleration。它采用内存列存储和改进的数据压缩技术、面向硬件特性的并行查询优化等技术加速分析处理性能。BLU Acceleration 与 DB2 构成双引擎,传统数据库引擎采用磁盘行存储表结构,提供事务处理能力;BLU Acceleration 引擎面向列存储,提供高性能分析处理能力。

图 17.8(c)为 Microsoft 公司的 SQL Server 产品结构。SQL Server 在传统磁盘行存储引擎的基础上增加了 Hekaton 内存行存储引擎,以加速联机事务处理性能;还增加了列存储索引,以加速联机分析处理性能。列存储索引可用于内存基本表,支持 B+树索引及数据同步更新,通过 SIMD 优化及批量处理技术提高查询性能。SQL Server 2019 CTP 2.1 支持基于 Intel Optane DC PMEM 非易失性内存的 Hybrid Buffer Pool 技术,通过直接对非易失性内存中数据的访问,消除数据从磁盘向 DRAM 缓冲区加载的代价。

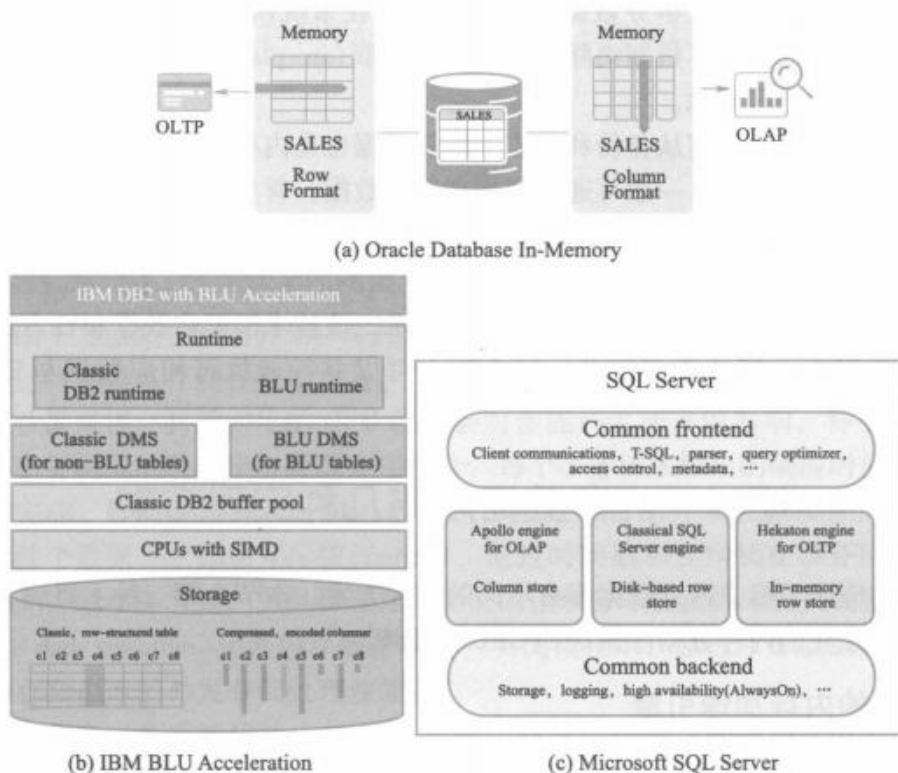


图 17.8 混合双/多引擎结构数据库产品示例

随着大内存、多核处理器逐渐成为主流的计算平台,传统的关系数据库系统正经历着从磁盘数据库到内存数据库的升级。内存计算的高性能进一步提高了实时分析处理能力,推动了事务处理与分析处理的融合技术,提高了分析处理的数据实时性。

17.4.2 独立的内存数据库系统

这种方案对应独立设计的内存数据库系统，代表性的数据库系统包括 SAP HANA、Vector (VectorWise)、VoltDB 等。

SAP HANA 内存数据库是一个集成事务处理与分析处理负载于一体的高性能内存数据库系统。其列存储引擎通过面向多核处理器、SIMD 指令、Cache、数据压缩和大内存的优化技术最大化数据库内核的并行处理能力，事务处理引擎采用适合事务处理的存储结构，数据库支持在操作数据上同时执行事务处理与分析处理任务。HANA 通过列存储数据压缩技术提高内存利用率，事务处理数据则采用简单的数据压缩方式，通过主存储和 delta 列存储两种存储模型分别优化 OLTP 与 OLAP 负载。当前，有研究将 NVRAM 作为内存数据库的新型非易失性存储，利用其大容量、低成本、接近内存访问性能的特点进一步提高内存数据库的性价比，提高内存数据库在重启时的数据加载性能。

Vector (VectorWise) 是荷兰 CWI 研究院在 MonetDB/X100 的基础上于 2008 年推出的商业化内存分析型数据库产品，通过与 INGRES 数据库结合，由 Action 公司于 2010 年推出 VectorWise 1.0 版本。该数据库的顶层采用 INGRES 架构，提供数据库管理、数据库连接、查询解析和基于代价的优化模型等功能，查询执行引擎和存储引擎采用 MonetDB/X100 模块，以列存储、CPU 效率高的轻量数据压缩、向量处理、多核并行处理等技术提高性能。在 VectorWise 1.0 分析数据库中具有 Hadoop 集成功能，由 Hadoop MapReduce 提供海量数据处理功能，并由 VectorWise 提供高性能大数据 SQL 引擎。

VoltDB 是一个基于 SN 架构，支持完全 ACID 特性的内存数据库。VoltDB 使用水平扩展技术增加 SN 集群中数据库的节点数量，将数据和数据上的处理相结合，分布在 CPU 核上作为虚拟节点。每个单线程分区作为一个自治的查询处理单元，消除并发控制代价。分区透明地在多个节点中进行分布，节点故障时自动由复制节点接替其处理任务。VoltDB 采用快照技术实现持久性，快照是数据库在一个时间点完整的数据库复制，保存在磁盘上。VoltDB 将事务处理作为编译的存储过程调用，查询被分配到节点控制器串行执行，通过基于 CPU 核的分区机制最大化并行事务处理能力，以满足高通量事务处理需求。

17.4.3 GPU 数据库

随着 GPU 处理器技术的发展，以 NVIDIA GPGPU 为代表的加速器集成了大量的计算核心，通过 GPU 的高并发线程提供强大的并行数据访问和计算能力。GPU 处理器技术的发展为提升数据库性能提供了硬件支持，GPU 既可以加速传统的磁盘数据库系统，也可以作为全新的内存数据库的加速引擎。

PG-Strom 是磁盘数据库 PostgreSQL 的 GPU 加速引擎，它通过 GPU 代码生成器为 SQL 查询任务创建 GPU 上的执行代码，通过 GPU 强大的并行处理能力加速查询处理。PG-Strom 支持查询中 SCAN、JOIN、GROUP BY 等操作在 GPU 端的执行，还支持 SSD-to-GPU 模式的 GPU

直接存储访问技术。

OmniSci (MapD) 是一个基于 GPU 和 CPU 混合架构的内存数据库, 包含内存数据库模式的 CPU 版本与加速模式的 GPU 版本。OmniSci 通过将用户查询编译为 CPU 和 GPU 上执行的机器码提高查询性能, 通过向量化查询执行和 GPU 代码优化技术提高查询执行性能。查询执行时, CPU 负责查询解析, 与 GPU 计算并行执行。

传统数据库的基础硬件假设正在发生变化, 非易失性内存成为大容量、低成本、高性能的新内存, 改变传统数据库面向易失性内存而设计的日志、缓存、恢复等机制; 硬件加速器成为高性能计算 (high performance computing, HPC) 的主流平台, 也将成为高性能数据库的计算平台, 面向 GPU 架构及 CPU-GPU 异构计算平台的数据库也成为一个新的技术发展趋势。

17.5 内存数据库前沿技术与展望

硬件技术的发展为内存数据库提供了硬件基础, 以 GPU、FPGA 为代表的异构计算体系结构的发展成为高性能计算平台的主流技术。

从存储技术的发展来看, 大内存已成为高性能计算的主流配置, 非易失性内存 NVRAM 已进入产品化阶段, 高性能多层堆叠 SSD 缩小了传统内存与外部存储之间的性能差距, RDMA、NVLink (GPU 架构) 等数据传输通信技术降低了网络访问延迟, 异构存储提供了基于性能、成本的不同组合方案。

传统的数据库基于同构硬件架构而设计。异构硬件架构需要数据库扩展对不同硬件的支持, 将数据库的存储引擎及查询引擎与异构硬件进行优化匹配, 为用户提供满足不同性能与成本需求的解决方案。

1. 面向千核处理器的内存数据库技术

随着 ARM 处理器成为 TOP 500 新型高性能计算平台, 基于 ARM 处理器的千核处理器计算平台成为新的高性能内存数据库硬件平台。与 X86 处理器相比, ARM 处理器的核心集成度更高, 基于多路处理器的千核计算平台为高性能内存数据库提供了更加强大的并行计算能力, 也需要数据库在千核并行处理技术上对传统的并发控制和查询处理进行优化, 以更好地发挥 ARM 处理器的并行计算能力。数据库传统的并发控制技术在千核处理器平台上不能自动适应, 更高的并发度甚至可能降低事务处理的整体性能, 需要根据千核处理器的硬件特征优化并行控制算法, 也需要对传统的并发控制算法进行深入评估, 确定最优并行度。另一方面, 千核处理器在分析处理负载中提供了更强大的并行计算能力, 但需要通过面向 NUMA 结构优化多核并行算法性能, 最大化千核并行计算性能。

2. 面向异构内存的内存数据库技术

NVRAM 的出现使内存成为持久存储设备, 补足了内存数据库在事务处理 ACID 特性中长期缺失的 D 特性, 从而使内存数据库成为真正意义上的完整数据库系统。NVRAM 具有内存的访问特性, 但其物理结构决定了它在内存访问延迟、带宽等性能上与 DRAM 存在差距, 在优

化访问方式上也有所差异,不能从当前的内存数据库架构直接升级,需要面向 DRAM-NVRAM 的异构内存特性而优化设计内存数据库存储和查询引擎,将较小的但数据访问频繁的数据或中间数据存储在 DRAM,而将较大的、数据访问延迟要求低的数据存储在 NVRAM,优化两种不同容量、不同访问性能、不同成本的混合内存上的综合查询性能。当前,在 NVRAM 数据库技术研究中,基于混合内存的存储优化、索引优化、查询优化等是主要的研究方向,主要的优化策略是尽量减少 NVRAM 在读、写、访问粒度上相对 DRAM 的较大延迟,通过 DRAM 优化 NVRAM 上的存储访问性能。

3. 面向 GPU-GPU 异构计算平台的内存数据库技术

GPU 与 FPGA 作为硬件加速器可以显著提升特定的计算性能,基于硬件加速器的异构计算平台为高性能数据库提供了硬件基础。其中 GPU 数据库发展最为成熟,已有一系列 GPU 数据库产品成为新一代高性能数据库的代表性技术。从数据库引擎实现技术来看, GPU 数据库有两条主要的技术路线。

(1) GPU 内存数据库

随着 GPU 技术的发展,更多核心、更大显存容量、更高数据通道性能(NVLink 或 PCIe 4.0/5.0)、多卡集成能力等特性,使 GPU 成为新的内存数据库平台,提供对一定规模数据集(基于数据压缩和多卡扩展存储容量)的高性能查询处理能力,从而获得远超过基于 CPU 和 DRAM 的内存数据库查询处理性能。

GPU 数据库是完全面向 GPU 特定的硬件架构而设计的数据库系统,通过实时编译的 cuda 代码生成技术和面向 GPU 存储结构的优化技术提高查询算子性能,在 TBps 级显存带宽性能的支持下提供支持百亿记录的毫秒级查询响应性能。

(2) GPU 加速引擎

GPU 加速引擎技术面向较大数据集的查询处理任务,通过将适合 GPU(而不适合 CPU)的计算密集型负载分配给 GPU 以达到提高查询算子性能的目标。当前, CPU 与 GPU 之间的 PCIe 通道带宽性能显著低于内存带宽和 GPU 显存带宽,成为 GPU 加速技术最主要的性能瓶颈。除数据压缩、传输优化、计算与数据传输流水并行等优化技术之外,提高 GPU 计算的数据局部性也是重要的优化方法,即从查询负载中分离出数据量小、计算量大、计算输出结果小的计算密集型负载并提高计算的局部性是一个有效的解决方案。从当前 GPU 数据库实现技术来看,面向关系操作算子的 GPU 加速技术因实时数据传输代价而降低了整体性能加速收益,因此定制适合 GPU 本地计算的操作算子是 GPU 加速技术的基础。

在 GPU 数据库的研究中,一方面需要根据 GPU 不同于 CPU 的硬件结构特点定制和优化 GPU 上的查询算法性能,另一方面需要根据 GPU 显存容量优化 CPU-GPU 混合存储模型,优化 CPU 与 GPU 间低速 PCIe 通道的数据传输代价。

4. HTAP 数据库引擎技术

传统的磁盘数据库受磁盘 I/O 带宽的影响而性能较低,需要将事务处理与执行时间较长的分析处理划分为独立的事务处理系统和分析处理系统,分别采用不同存储模型的查询优化方

法。内存数据库极大地提升了事务处理系统和分析处理系统的性能,尤其使分析处理系统不依赖于存储和计算成本较高的物化视图、索引等机制,支持实时分析处理。内存数据库性能的提升推动了混合事务分析处理技术的发展,即将事务处理系统与分析处理系统集成在一个数据库引擎内,实现实时的事务与分析处理。混合事务分析处理面对两种不同特征的负载处理,需要在存储模型和查询处理两方面实现融合与资源分配。

当前的混合事务分析处理技术主要体现在存储模型融合技术上,通常在数据库内部采用适合事务处理的行存储和适合分析处理的列存储混合引擎,通过一定的优化机制,实现内部行存储数据向列存储数据的迁移和存储结构转换。混合事务分析处理系统实现了事务处理引擎与分析处理引擎在数据库系统内部的集成,由数据库系统自动完成事务处理与分析处理任务以及行列存储转换,为用户提供统一的数据库平台。在分布式系统中,一种代表性的混合事务分析处理技术是:利用多个数据副本和日志更新策略保持事务处理副本与分析处理副本的一致性,通过不同副本的并行处理保证事务处理与分析处理负载同时具有较高的性能。

混合存储引擎技术解决了事务处理系统与分析处理系统在数据存储模型上的统一,在高负载应用场景下,分析处理负载需要占用大量处理器资源执行长程查询处理(long-term query),短事务处理需要与分析处理竞争处理器资源,降低了查询响应时间。异构计算平台为混合事务分析处理提供了新的解决方案,集成的高性能硬件加速器适用于分析处理负载执行,从而为异构计算平台上的混合事务分析处理系统释放处理器资源用于事务处理,使事务处理和分析处理负载在处理器资源上最大化隔离,减少两种负载对处理器资源的争用,提高两种负载各自的处理性能。当前 GPU 数据库通常作为独立的联机分析处理数据库引擎,主要处理数据仓库负载,未来将成为新型的混合事务分析处理计算平台,实现内存事务处理与 GPU 分析处理的集成与融合,为用户提供统一的数据库应用平台。

本章小结

大容量内存、非易失性内存、闪存、多核 CPU、众核处理器、高性能网络传输等硬件技术的发展,为内存数据库提供了良好的平台。虽然内存价格相对于传统的磁盘仍然很高,但内存数据库的软件结构相对更简单,代码执行效率更高,不需要复杂的索引、物化视图等传统的数据库调优技术,具有更好的性价比。

内存事务处理数据库的软件技术比较成熟,已大量应用于金融、电信等实时响应性能要求较高的应用领域,成为企业核心业务事务处理的解决方案。随着内存硬件价格的不断下降,内存分析处理数据库成为内存计算新的技术增长点,一些商业内存数据库近年来取得了巨大的市场份额,传统数据库厂商 Oracle、IBM 和 Microsoft 等分别推出了内存数据库产品及解决方案,新兴的内存数据库产品 SAP HANA 及 VectorWise 等也迅速被高端数据库应用所采纳。应用案例表明,即使是一些大型企业,其核心分析数据也能够完全存储于内存或内存数据库集群中以支持高性能分析处理。

当前的服务器已能够支持 TB 级内存,在数据压缩技术的支持下,内存数据库平台能够支持数倍甚至数十倍于物理内存的大数据处理任务,成为大数据应用的有效解决方案。随着新硬件技术的不断发展,内存数据库技术也将随之而不断发展。非易失性内存技术的成熟进一步为内存数据库技术的发展提供了硬件基础,推动内存数据库成为大数据实时分析处理的重要平台。新型处理器技术的发展推动高性能计算平台走向异构化,如何在异构计算平台上进一步提高内存数据库的性能和处理能力是未来的一个重要研究方向。

限于篇幅,本章许多技术仅作概要论述,感兴趣的读者可以阅读本章列出的参考文献。

习 题 17

1. 内存数据库和磁盘数据库有什么区别?
2. 磁盘数据库与内存数据库各适用于哪些应用场景?
3. 试述内存列存储模型的结构特点、在联机事务处理与联机分析处理应用场景中行存储结构和列存储结构的优缺点,分析存储模型与应用模式的关系。
4. 假设订单表 ORDERS 包含 16 个 INT 类型(4 B)的数据列,表中记录为 10 亿条,分别采用磁盘行存储结构和内存列存储结构。假设磁盘访问带宽为 100 MBps,内存带宽为 200 GBps。查询需要扫描表中的任意 3 个数据列,试估算在两种不同存储结构下的表扫描时间。
5. 试述内存数据库和硬件的相关性。多核 CPU、GPU、非易失性存储器等新硬件对内存数据库的设计有哪些影响,带来哪些性能收益?
6. 以内存哈希连接算法为例,试述在多核 CPU 平台和 GPU 平台如何根据硬件特性进行优化设计。

参考文献 17

- [1] 张延松,王珊.内存数据库技术与实现[M].北京:高等教育出版社,2016.
- [2] RAO J, ROSS K A. Making B+ trees cache conscious in main memory[C]. Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, 2000: 475-486.
- [3] LEE I H, SHIM J, LEE S G, et al. CST-trees: cache sensitive T-trees[C]. Proceeding of the 12th International Conference on Database Systems for Advanced Applications(DASFAA 2007), 2007: 398-409.
- [4] MANEGOLD S, BONCZ P, NES N, et al. Cache-conscious radix-decluster projections[C]. Proceedings of the 30th International Conference on Very Large Data Base, 2004, 30: 684-695.
- [5] BALKESSEN C, TEUBNER J, ALONSO G, et al. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware[C]. Proceedings of the 29th IEEE International Conference on Data Engineering, 2013: 362-373.
- [6] KEMPER A, NEUMANN T, FINIS J, et al. Transaction processing in the hybrid OLTP&OLAP main-memory database system HyPer[J]. IEEE Data Engineering Bulletin, 2013, 36(2): 41-47.
- [7] SIKKA V, FÄRBER F, LEHNER W, et al. Efficient transaction processing in SAP HANA database: the end of a column store myth[C]. Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data,

2012; 731–742.

[8] HE B S, LU M, YANG K, et al. Relational query coprocessing on graphics processors[J]. ACM Transactions on Database Systems, 2009, 34(4): 1–39.

[9] HE B S, Yu J X. High-throughput transaction executions on graphics processors[C]. Proceedings of the VLDB Endowment, 2011, 4(5): 314–325.

[10] ZUKOWSKI M, BONCZ P A, NES N, et al. MonetDB/X100 –a DBMS in the CPU cache[J]. IEEE Data Engineering Bulletin, 2005, 28(2): 17–22.

[11] NEUMANN T, LEIS V, et al. Compiling database queries into machine code[J]. IEEE Data Engineering Bulletin, 2014, 37(1): 3–11.

[12] BALKESSEN C, ALONSO G, TEUBNER J, et al. Multi-core, main-memory joins: sort vs. hash revisited [C]. Proceedings of the VLDB Endowment, 2013, 7(1): 85–96.

[13] AREFYEVA I, BRONESKE D, CAMPERO G, et al. Memory management strategies in CPU/GPU database systems: a survey[C]. International Conference: Beyond Databases, Architectures and Structures (BDAS'18), 2018: 128–142.

[14] HALSTEAD R J, ABSALYAMOV I, NAJJAR W A, et al. FPGA-based multithreading for in-memory hash joins[C]. Proceedings of the 7th Biennial Conference on Innovative Data Systems Research (CIDR'15), 2015.

[15] CHENG X T, HE B, S LU M, et al. Efficient query processing on many-core architectures: a case study with Intel Xeon Phi processor[C]. Proceedings of the 2016 International Conference on Management of Data, 2016: 2081–2084.

[16] ABADI D, AILAMAKI A, ANDERSEN D, et al. The seattle report on database research[J]. ACM SIGMOD Record, 2019, 48(4): 44–53.

[17] SHANBHAG A, MADDEN S, YU X Y. A study of the fundamental performance characteristics of GPUs and CPUs for database analytics[C]. Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD'20), 2020: 1617–1632.

[18] YU X Y, YOUILL M, WOICIK M, et al. PushdownDB: accelerating a DBMS using S3 computation[C]. Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE), 2020: 1802–1805.

[19] LU Y, YU X Y, CAO L, et al. Aria: a fast and practical deterministic OLTP database[C]. Proceedings of the VLDB Endowment, 2020, 13(12): 2047–2060.

[20] LU B T, HAO X P, WANG T Z, et al. Dash: scalable hashing on persistent memory[C]. Proceedings of VLDB Endowment. 2020, 13(8): 1147–1161.

[21] HAO X P, LERSCH L, WANG T Z, et al. PiBench online: interactive benchmarking of persistent memory indexes[C]. Proceedings of VLDB Endowment, 2020, 13(12): 2817–2820.

[22] HE Y J, LU J C, WANG T Z. CoroBase: coroutine-oriented main-memory database engine[Z]. 2020, CoRR abs/2010.15981.

[23] LIU J H, CHEN S M, WANG L J. LB+Trees: optimizing persistent index performance on 3DXPoint memory [C]. Proceedings of VLDB Endowment, 2020, 13(7): 1078–1090.

本章简要而系统地介绍区块链技术，主要包括区块链的概念与工作原理、区块链的发展进程、区块链系统的技术架构与关键技术、区块链与数据库的对比与融合等。

区块链是数据块的链接列表，是存储数据的一种新型数据结构。直观地看，它构成了数据更新的日志。

本章的目的不仅仅在于介绍区块链技术，而是力求从数据库视角出发，将区块链技术和数据库技术相结合，使其能够相互借鉴。在数据库系统中能够借鉴和应用区块链的去中心化技术、去信任理念和防篡改技术，使数据库应用更加安全可靠。同时，区块链能够从数据库查询优化和存储管理等技术中得到启发，进一步扩展其查询功能，提高事务处理效率，使得区块链的应用更加广泛深入。

18.1 区块链的概念与工作原理

区块链概念诞生至今仅十多年，目前已发展成为时下技术热点之一。区块链作为一种新型的分布式数据存储与处理系统，已成为数据库领域重要的研究和应用方向。

2016 年 12 月，区块链首次作为战略性前沿技术写入《“十三五”国家信息化规划》，我国将区块链定位为核心技术自主创新的重要突破口。

本章内容参考了中国信息通信研究院 (China Academy of Information and Communications Technology, CAICT, 简称中国信通院) 与可信区块链推进计划 (Trusted Blockchain Initiatives, TBI) 发布的若干版本白皮书^[1-3]，以及国内外相关文献与专家报告等。

区块链诞生时日毕竟尚短，技术成熟度有限。除比特币 (Bitcoin) 和以太坊 (Ethereum) 以外，区块链技术目前还缺乏大型应用实例的支撑，其标准体系尚未完备搭建，很多提法尚处于探讨研究阶段。本章内容和观点需要随着区块链技术和应用的不断发展进行修正和完善。

互联网是一场革命，它解决了信息的高效和廉价传输，但是它也带来了新问题，因为在网络上复制、传播、篡改一条信息的成本几乎为零，网络信息的可信度大打折扣。在货币领域，尽管电子货币带给人类极大便利，但目前的电子货币还必须依赖大量的第三方中介机构才能实